

Linux Programming

Unit -6

Contents

- Kernel support for semaphore
- Unix System V APIs for Semaphores
- Kernel support for shared memory
- Unix System V APIs for Shared memory

Semaphore Definition

- A semaphore is a data structure that is shared by several processes. Semaphores are most often used to synchronize operations (to avoid race conditions) when multiple processes access a common, non-shareable resource.
- By using semaphores, we attempt to avoid other multi-programming problems such as:
 - Starvation
 - » Occurs when a process is habitually denied access to a resource it needs.
 - Deadlock
 - » Occurs when two or more processes each hold a resource that the other needs while waiting for the other process to release its resource.

Semaphore Definition

- To indicate a process has gained access to the resource, the process decrements the semaphore.
- For events to progress correctly, the test and decrement operation on the semaphore must be
- atomic (i.e., noninterruptible/indivisible).
- There are two kinds of Semaphores:
 - Binary semaphores
 - » Control access to a single resource, taking the value of 0 (resource is in use) or 1 (resource is available).
 - Counting semaphores
 - » Control access to multiple resources, thus assuming a range of nonnegative values.

Semaphores

- Semaphores provide a method to synchronize the execution of multiple processes.
- A semaphore is a counter used to provide access to a shared data object for multiple processes
- A process can also use multiple semaphore sets.
- Semaphores are frequently used along with shared memory to establish an elaborate method for interprocess communication.
- To obtain a shared resource, a process needs to do the following:
 1. Test the semaphore that controls the resource.
 2. If the value of the semaphore is positive, the process can use the resource. In this case, the process decrements the semaphore value by 1, indicating that it has used one unit of the resource.
 3. Otherwise, if the value of the semaphore is 0, the process goes to sleep until the semaphore value is greater than 0. When the process wakes up, it returns to step 1.

Semaphores

Creating and Accessing Semaphore Sets

- Before a semaphore set can be used, it must be created.
- The creation of the semaphore set generates a unique data structure that the system uses to identify and manipulate the semaphores.
- A conceptual arrangement of a system semaphore structure for a newly allocated set of three semaphores is shown in Figure

Semaphores

Creating and Accessing Semaphore Sets

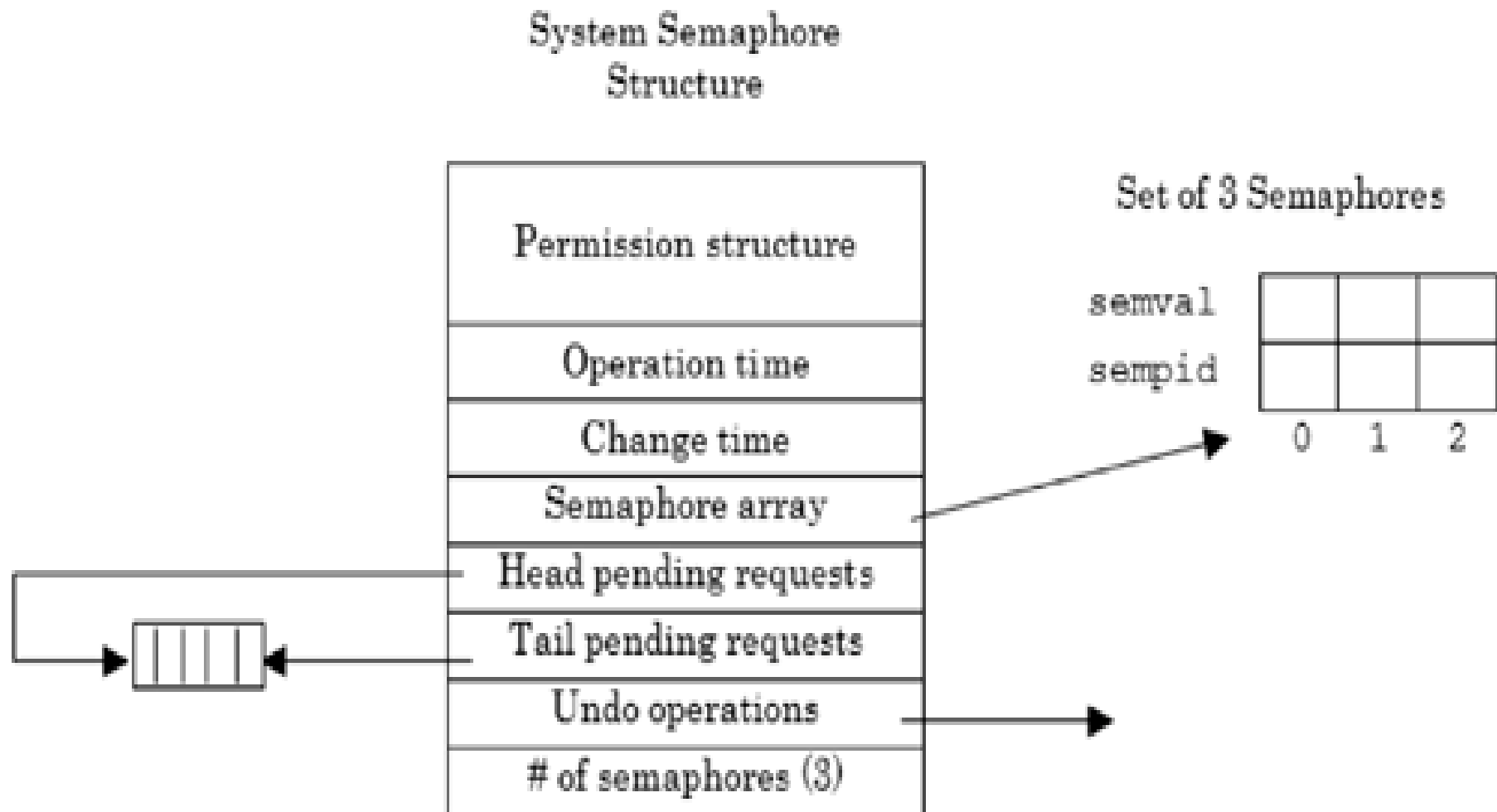


Fig. Data structures for a set of three semaphores.

Semaphores

Unix System V Semaphore APIs provide the following functions

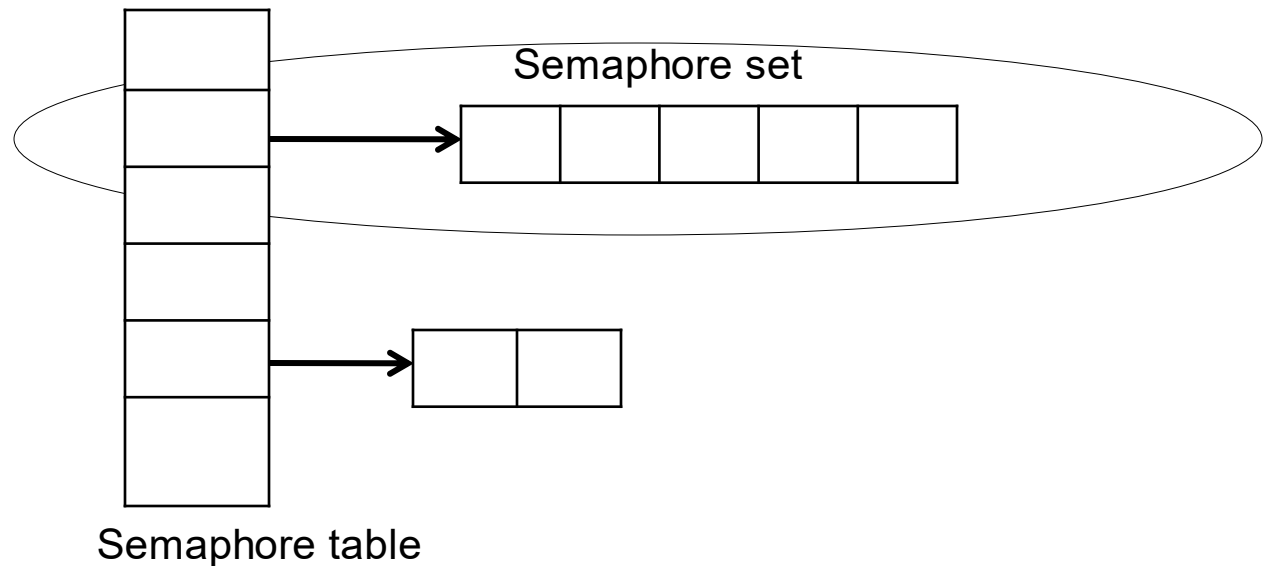
- Create a semaphore set
- “Open” a semaphore set and get a descriptor to reference the set
- Increase or decrease the integer values of one more semaphores in a set
- Query the values of one or more semaphores in a set
- Query or set control data of a semaphore set

Kernel Support for Semaphores

- In UNIX System V.3 and V.4, there is a semaphore table in the kernel address space that keep track of all semaphore sets created in the system.
- Each entry of the semaphore table stores the following data for one semaphore set:
 - A name that is an integer ID key assigned by the process which created the set. Other processes may specify this key to “open” the set and get a descriptor for future access of the set.
 - The creator user ID and group ID. A process whose effective user ID matches a semaphore set creator user ID may delete the set and also change control data of the set.
 - The assigned owner user ID and group ID. These normally same as the creator user and group ID, but a creator process can set these values to assign a different owner and group membership for the set.
 - Read-write access permission of the set for owner, group members, and others.
 - The number of semaphores in the set.
 - The time when the last process changed one or more semaphore values.
 - The time when the last process changed the control data of the set.
 - A pointer to an array of semaphores.

Kernel Support for Semaphores

- Semaphores in a set are referenced by array indices, such that the first semaphore in the set has an index of zero; the second semaphore has an index of 1; and so on.
- Furthermore, each semaphore stores the following data:
 - The semaphore's value
 - The process ID of the last process that operated on the semaphore
 - The number of processes that are currently blocked pending the increase of semaphore value
 - The number of processes which are currently blocked pending the semaphore's value becoming zero.



Kernel data structure for semaphores

Kernel Support for Semaphores

previous figure depicts the Unix kernel data structure for semaphores.

- Like messages semaphores are stored in a kernel address space and are persistent, despite their creator process's termination.
- Each semaphore set has assigned owner, and only processes that have super user, set creator, or assigned owner privileges may delete the set or change its control data.
- If a semaphore set is deleted, any processes that are blocked at that time due to the semaphores are awakened by the kernel- the system calls they invoked are aborted and return a-1 failure status.

Finally, there are several system-imposed limits on the manipulation of Semaphores. These limits are defined in the `<sys/sem.h>` header:

System limit	Meaning
SEMMNI	The max. no. of semaphore sets that may exist at any given time in a system.
SEMMNS	The max. no. of semaphores in all sets that may exist in a system at any one time.
SEMMSL	The maximum number of semaphores allowed per set
SEMOPM	The max. no. of semaphores in a set that may be operated on at any time

Kernel Support for Semaphores

The effects of these system-imposed limits on processes are :

- If a process attempts to create a new semaphore set that causes either the SEMMNI or the SEMMNS limit to be exceeded, the process will be blocked until one or more existing sets are deleted by a process.
- If a process attempts to create a semaphore set with more than SEMMSL semaphores, the system call fails.
- If a process attempts to operate on more than SEMOPM semaphores in a set in one operation, the system call fails.

Unix System V APIs for Semaphores

- The `<sys/ipc.h>` header defines a *struct ipc_perm* data type, which stores the user ID, group ID, assigned name key, and the read-write permission of a semaphore set.
- The semaphore table entry data type is *struct semid_ds*, which is defined in the `<sys/sem.h>` header . The data fields of the data stored are:

Data field

Data Stored

sem_perm

Data stored in a *struct ipc_perm* record

sem_nsems

Number of semaphores in the set

sem_base

Pointer to an array of semaphores

sem_otime

Time when last process operated on
semaphores

sem_ctime

Time when last process changed control
data of the set.

Unix System V APIs for Semaphores

In addition to the above, the *struct sem* data type , as defined in the <sys/sem.h> header, defines the data stored in a semaphore:

Data field

semval

sempid

semncnt

semzcnt

Data Stored

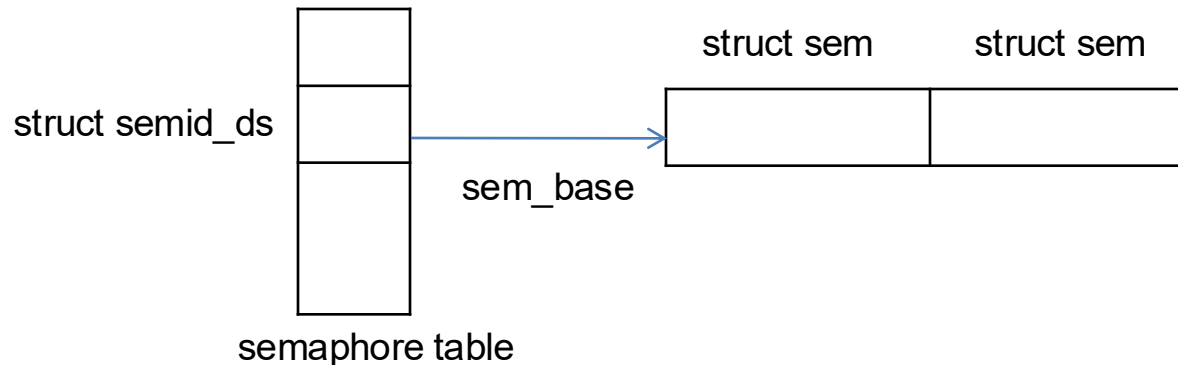
Current semaphore's integer value

Process ID of the last process that operated on the semaphore

Number of processes that are blocked waiting for the semaphore's value to be increased.

Number of processes that are blocked waiting for the semaphore's value to become zero

Illustrates uses of the aforementioned structures in the semaphore table and semaphore records.



Data type of the semaphore table and semaphore record

Unix System V APIs for Semaphores

Semaphore API

semget
semop
semctl

Usages

Open and create, if needed, a semaphore set
Change or query semaphore value
Query or change control data of a semaphore set or delete a set

The header file needed for the semaphore APIs are:

```
#include<sys/types.h>  
#include<sys/sem.h>
```

1) **semget**

The function prototype of the *semget* API is:

```
#include<sys/types.h>  
#include<sys/ipc.h>  
#include<sys/sem.h>  
int semget ( key_t key, int num_sem, int flag );
```

Unix System V APIs for Semaphores

- This function “opens” a semaphore set whose key ID is given in the *key* argument. The function returns a nonnegative integer descriptor that can be used in the other semaphore APIs to change or query semaphore value and to query and/or set control data for the semaphore set.
- If the value of the *key* argument is a positive integer, the API attempts to open a semaphore set whose key ID matches that value.
- However, if the key value is the manifested constant `IPC_PRIVATE`, the API allocates a new semaphore set to be used exclusively by the calling process.
- Specifically, the “private” semaphores are usually allocated by a parent process which then forks one or more child processes.

Unix System V APIs for Semaphores

Creating and Accessing Semaphore Sets

- To create a semaphore or gain access to one that exists, the `semget` system call is used. (Table 6.1)

Include File(s)	<code><sys/types.h></code> <code><sys/ipc.h></code> <code><sys/sem.h></code>	Manual Section	2	
Include File(s)	<code><sys/types.h></code> <code><sys/ipc.h></code> <code><sys/sem.h></code>	Manual Section	2	
Return	Success		Failure	Sets <code>errno</code>
	The semaphore identifier		-1	Yes

Table 61. Summary of the `semget` System Call

Unix System V APIs for Semaphores

Creating and Accessing Semaphore Sets

- The `semget` system call takes three arguments:
 - The first argument, *key*, is used by the system to generate a unique semaphore identifier.
 - The second argument, *nsems*, is the number of semaphores in the set.
 - The third argument, *semflg*, is used to specify access permission and/or special creation conditions.

Unix System V APIs for Semaphores

Creating and Accessing Semaphore Sets

#	Constant	Error Message	Explanation
2	EOENT	No such file or directory	Semaphore identifier does not exist for this key, and IPC_CREAT was not set.
12	ENOMEM	Cannot allocate memory	Insufficient system memory to allocate the semaphore set.
13	EACCES	Permission denied	Semaphore identifier exists for this key, but requested operation is not allowed by current access permissions.
17	EEXIST	File exists	Semaphore identifier exists for this key, but the flags IPC_CREAT and IPC_EXCL are both set.
28	ENOSPC	No space left on device	System-imposed limit (SEMMNI) for the number of semaphore sets or systemwide maximum number of semaphores (SEMMNS) has been reached.
43	EIDRM	Identifier removed	Specified semaphore set is marked for removal.

Table 6.2. semget Error Messages.

Unix System V APIs for Semaphores

Semaphore Control

- The `semctl` system call allows the user to perform a variety of generalized control operations on the system semaphore structure, on the semaphores as a set, and on individual semaphores. (Table 6.3)

Include File(s)	<sys/types.h> <sys/ipc.h> <sys/sem.h>	Manual Section	2
Summary	int semctl(int semid, int semnum, int cmd, union semun arg);		
Return	Success	Failure	Sets errno
	0 or the value requested	-1	Yes

Table 6.3. Summary of the semctl System Call

Unix System V APIs for Semaphores

Semaphore Control

- The semctl system call takes four arguments:
 - The first argument, semid, is a valid semaphore identifier that was returned by a previous semget system call.
 - The second argument, semnum, is the number of semaphores in the semaphore set (array), 0 means this number (index) is not relevant.
 - The third argument to semctl, cmd, is an integer command value. the cmd value directs semctl to take one of several control actions. Each action requires specific access permissions to the semaphore control structure.

Unix System V APIs for Semaphores

Semaphore Control

- The fourth argument to `semctl`, `arg`, is a union of type `semun`. Given the action specified by the preceding `cmd` argument, the data in `arg` can be one of any of the following four values:
 - » An integer already was set in the `val` member of `sem_union` that used with `SETVAL` to indicate a change of specific value for a particular semaphore within the semaphore set.
 - » A reference to a `semid_ds` structure where information is returned when `IPC_STAT` or `IPC_SET` is specified.
 - » A reference to an array of type unsigned short integers; the array is used either to initialize the semaphore set or as a return location when specifying `GETALL`.
 - » A reference to a `seminfo` structure when `IPC_INFO` is requested.

Unix System V APIs for Semaphores

Semaphore Control

- If `semctl` fails, it returns a value of `-1` and sets `errno` to indicate the specific error. (Table 6.4.)

#	Constant	Error Message	Explanation
1	EPERM	Operation not permitted	Value for <code>cmd</code> is <code>IPC_RMID</code> or <code>IPC_SET</code> and the calling process is not the owner or superuser.
13	EACCES	Permission denied	The requested operation is not allowed by the current access permissions for this process.
14	EFAULT	Bad address	The fourth argument to <code>semctl</code> contains a reference to an illegal address (the union <code>semun</code> may not have been declared).
22	EINVAL	Invalid argument	• The semaphore identifier is invalid. • The number of semaphores specified is less than 0 or greater than the number in the semaphore set. • The value for <code>cmd</code> is invalid. • The value for <code>cmd</code> is <code>IPC_SET</code>, but the value for <code>sem_perm.uid</code> or <code>sem_perm.gid</code> is invalid.

Table 6.4. semctl Error Messages.

Unix System V APIs for Semaphores

Semaphore Control

#	Constant	Error Message	Explanation
34	ERANGE	Numerical result out of range	The value for cmd is SETVAL or SETALL, and the value to be assigned is greater than the system maximum or less than 0.
43	EIDRM	Identifier removed	Specified semaphore set is marked for removal.

Table 6.4. semctl Error Messages.

Unix System V APIs for Semaphores

Semaphore Control Details

- The following cmd values cause semctl to act upon the system semaphore structure
 - IPC_STAT
 - » Return the current values of the semid_ds structure for the indicated semaphore identifier. The returned information is stored in a user-generated structure referenced by the fourth argument to semctl. To specify IPC_STAT, the process must have read permission for the semaphore set associated with the semaphore identifier.

Unix System V APIs for Semaphores

Semaphore Control Details

- `IPC_SET`
 - » Modify a restricted number of members in the `semid_ds` structure. The members `sem_perm.uid`, `sem_perm.gid` and `sem_perm.mode` can be changed if the effective ID of the accessing process is that of the superuser or is the same as the ID value stored in `sem_perm.cuid` or `sem_perm.uid`. To make these changes, a structure of the type `semid_ds` must be allocated. The appropriate members' values are then assigned, and a reference to the modified structure is passed as the fourth argument to the `semctl` system call.
- `IPC_RMID`
 - » Remove the semaphore set associated with the semaphore identifier.

Unix System V APIs for Semaphores

Semaphore Control Details

- The following cmd values cause semctl to act upon the entire set of semaphores:
 - GETALL
 - » Return the current values of the semaphore set. The values are returned via the array reference passed as the fourth argument to semctl. The user is responsible for allocating the array of the proper size and type prior to passing its address to semctl. Read permission for the semaphore set is required to specify GETALL. When specifying GETALL, the argument semnum is ignored.

Unix System V APIs for Semaphores

Semaphore Control Details

➤ SETALL

- » Initialize all semaphores in a set to the values stored in the array referenced by the fourth argument to `semctl`. Again, the user must allocate the initializing array and assign values prior to passing the address of the array to `semctl`. The process must have alter access for the semaphore set to use SETALL. When specifying SETALL, the `sem_ctime` member of the system semaphore data structure is updated.

Unix System V APIs for Semaphores

Semaphore Control Details

- The last set of semctl cmd values acts upon individual semaphores or upon specific members in the semid_ds structure. All of these commands require read permission except for SETVAL, which requires alter permission:
 - GETVAL
 - » Return the current value of the individual semaphore referenced by the value of the semnum argument.
 - SETVAL
 - » Set the value of the individual semaphore referenced by the semnum argument to the value specified by the fourth argument to semctl.

Unix System V APIs for Semaphores

Semaphore Control Details

- GETPID
 - » Return the PID from the sem_perm structure within the semid_ds structure.
- GETNCNT
 - » Return the number of processes waiting for the semaphore referenced by the semnum argument to increase in value.
- GETZCNT
 - » Return the number of processes waiting for the semaphore referenced by the semnum argument to become 0.

Unix System V APIs for Semaphores

Semaphore Operations

- Additional operations on individual semaphores are accomplished by using the semop system call. (Table 6.5.)

Include File(s)	<sys/types.h> <sys/ipc.h> <sys/sem.h>	Manual Section	2
Summary	int semop(int semid, struct sembuf*sops, unsigned nsops);		
Return	Success	Failure	Sets errno
	0	-1	Yes

Table 6.5. Summary of the semop System Call

Unix System V APIs for Semaphores

Semaphore Operations

- The semop system call takes three arguments:
 - The first argument, semid, is a valid semaphore identifier that was returned by a previous successful semget system call.
 - The second argument, sops, is a reference to the base address of an array of semaphore operations that will be performed on the semaphore set associated with by the semid value.
 - The third argument, nsops, is the number of elements in the array of semaphore operations.

Unix System V APIs for Semaphores

Semaphore Operations

#	Constant	Error Message	Explanation
4	EINTR	Interrupted system call	While in a wait queue for the semaphore, a signal was received by the calling process.
7	E2BIG	Argument list too long	The value for nsops is greater than the system limit.
11	EAGAIN	Resource temporarily unavailable	The requested operation would cause the calling process to block, but IPC_NOWAIT was specified.
12	ENOMEM	Cannot allocate memory	The limit for number of processes requesting SEM_UNDO has been exceeded.
13	EACCES	Permission denied	The requested operation is forbidden by the current access permissions.
14	EFAULT	Bad address	The value for sops references an illegal address.
22	EINVAL	Invalid argument	• The semaphore identifier is invalid. • The number of semaphores requesting SEM_UNDO is greater than the system limit.

Table 6.6. semop Error Messages.

Unix System V APIs for Semaphores

Semaphore Operations

#	Constant	perorr Message	Explanation
27	EFBIG	File too large	The value for sem_num is < 0 or >= to the number of semaphores in the set.
34	ERANGE	Numerical result out of range	The requested operation would cause the system semaphore adjustment value to exceed its limit.
43	EIDRM	Identifier removed	The semaphore set associated with semid value has been removed.

Table 6.6. semop Error Messages.

Unix System V APIs for Semaphores

Semaphore Operation Details

- When the `sem_op` value is negative, the process specifying the operation is attempting to decrement the semaphore.
- The decrement of the semaphore is used to record the acquisition of the resource affiliated with the semaphore.
- When a semaphore value is to be modified, the accessing process must have alter permission for the semaphore set.
(Table 6.7.)

Unix System V APIs for Semaphores

Semaphore Operation Details

Condition	Flag Set	Action Taken by semop
$\text{semval} \geq \text{abs}(\text{semop})$		Subtract $\text{abs}(\text{sem_op})$ from semval .
$\text{semval} \geq \text{abs}(\text{semop})$	SEM_UNDO	Subtract $\text{abs}(\text{sem_op})$ from semval and update the undo counter for the semaphore.
$\text{semval} < \text{abs}(\text{semop})$		Increment semncnt for the semaphore and wait (block) until • $\text{semval} \geq \text{abs}(\text{semop})$, then adjust semncnt and subtract as noted in the previous two rows of table. • semid is removed, then return -1 and set errno to EIDRM. • A signal is caught, then adjust semncnt and set errno to EINTR.
$\text{semval} < \text{abs}(\text{semop})$	IPC_NOWAIT	Return -1 immediately and set errno to EAGAIN .

Table 6.7 Actions Taken by semop when the Value for sem_op is Negative.

Unix System V APIs for Semaphores

Semaphore Operation Details

- When the `sem_op` value is positive, the process is adding to the semaphore value. The addition is used to record the return (release) of the resource affiliated with the semaphore.
- Again, when a semaphore value is to be modified, the accessing process must have alter permission for the semaphore set.

(Table 6.8.)

Condition	Flag Set	Action Taken by <code>semop</code>
		Add <code>sem_op</code> to <code>semval</code> .
	<code>SEM_UNDO</code>	Add <code>sem_op</code> to <code>semval</code> and update the undo counter for the semaphore.

Table 6.8. Actions Taken by `semop` when the Value for `sem_op` is Positive.

Unix System V APIs for Semaphores

Semaphore Operation Details

- When the `sem_op` value is zero, the process is testing the semaphore to determine if it is at 0.
- When a semaphore is at 0, the testing process can assume that all the resources affiliated with the semaphore are currently allocated (in use).
- For a semaphore value to be tested, the accessing process must have read permission for the semaphore set.
- (Table 6.9.)

Unix System V APIs for Semaphores

Semaphore Operation Details

Condition	Flag Set	Action Taken by semop
semval == 0		Return immediately.
semval != 0	IPC_NOWAIT	Return -1 immediately and set errno to EAGAIN.
semval != 0		Increment semzcnt for the semaphore and wait (block) until • semval == 0, then adjust semzcnt and return. • semid is removed, then return -1 and set errno to EIDRM. • A signal is caught, then adjust semzcnt and set errno to EINTR.

Table 6.9. Actions Taken by semop when the Value for sem_op is Zero.

Unix System V APIs for Semaphores

Deleting Semaphores

- The `semctl` command with `IPC_RMID` removes the semaphore in the program
- The command `"ipcs -s"` will list all semaphores on a system.
- The command `"ipcrm -s {semid}"` will delete a semaphore on system prompt.
- To delete all semaphores you have authority over, you can use this on system;

```
for semid in `ipcs -s | cut -d\ -f2`; do ipcrm -s $semid; done
```


Shared Memory

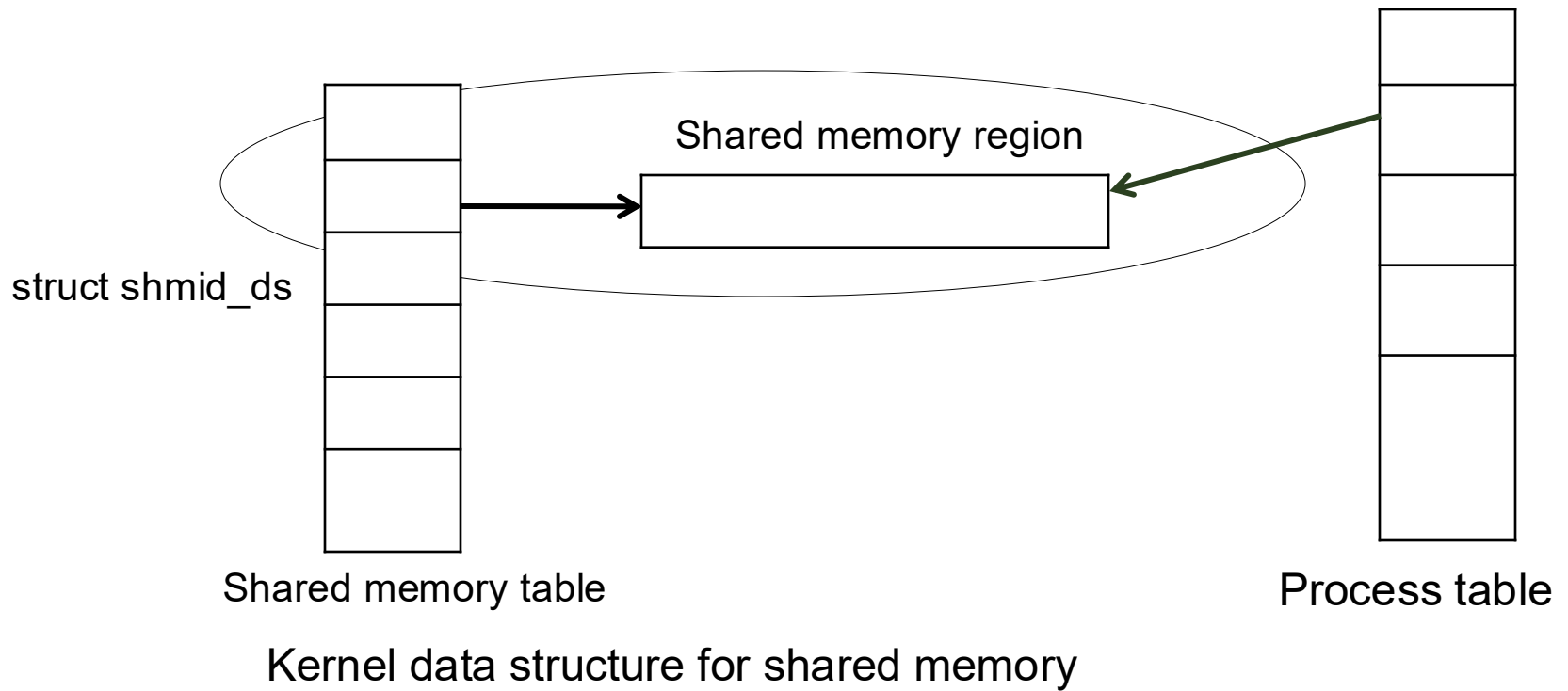
- Shared memory allows multiple processes to share virtual memory space.
- This is the fastest but not necessarily the easiest (synchronization-wise) way for processes to communicate with one another.
- In general, one process creates or allocates the shared memory segment.
- The size and access permissions for the segment are set when it is created.
- The process then attaches the shared segment, causing it to be mapped into its current data space.

Shared Memory

- If needed, the creating process then initializes the shared memory.
- Once created, and if permissions permit, other processes can gain access to the shared memory segment and map it into their data space.
- Each process accesses the shared memory relative to its attachment address.
- While the data that these processes are referencing is in common, each process uses different attachment address values.
- For each process involved, the mapped memory appears to be no different from any other of its memory addresses.

Kernel Support for Shared Memory

- The kernel keeps track of all the shared memory regions created in a system.



Kernel Support for Shared Memory

- For each shared memory region, the kernel maintains the following information:
 - i) Owner and the access permission for this regions.
 - ii) A pointer in the kernel.
 - iii) The size of shared memory region in bytes.
 - iv) The number of times the region is locked.
 - v) The process ID of the last processes that created and manipulated the regions.
 - vi) The number of processes attached to the region and its related information.
 - vii) The time at which the last process attached to and detached from the region.
 - viii) The time at which the last process changed control data of the region.

Kernel Support for Shared Memory

- The shared memory regions created using shmget system call.
- The shmat and shmdt attaches and detaches region from the address space of calling process respectively.
- There are several system limits that affects shared memory. These limits are defined in the “sys/shm.h” header file.

System-limit	Meaning	Value
SHMMNI	The maximum no. of shared memory regions allowed in a system	100
SHMSEG	The maximum no. of shared memory regions allowed for any process	6
SHMMIN	The maximum no. of bytes allowed in a shared memory region.	1
SHMMAX	The maximum no. of bytes allowed in a shared memory region	131,072

Kernel Support for Shared Memory

- The shared memory regions created using shmget system call.
- The shmat and shmdt attaches and detaches region from the address space of calling process respectively.
- There are several system limits that affects shared memory. These limits are defined in the “sys/shm.h” header file.

System-limit	Meaning	Value
SHMMNI	The maximum no. of shared memory regions allowed in a system	100
SHMSEG	The maximum no. of shared memory regions allowed for any process	6
SHMMIN	The maximum no. of bytes allowed in a shared memory region.	1
SHMMAX	The maximum no. of bytes allowed in a shared memory region	131,072

Creating a Shared Memory Segment

- The `shmget` system call is used to create the shared memory segment and generate the associated system data structure or to gain access to an existing segment.
- The shared memory segment and the system data structure are identified by a unique shared memory identifier that the `shmget` system call returns.

(Table 8.1)

Creating a Shared Memory Segment

Include File(s)	<code><sys/ipc.h></code> <code><sys/shm.h></code>	Manual Section	2
Summary	<code>int shmget(key_t key, int size,int shmflg);</code>		
Return	Success	Failure	Sets errno
	Shared memory identifier.	-1	Yes

Table 8.1. Summary of the shmget System Call.

Creating a Shared Memory Segment

- The `shmget` system call creates a new shared memory segment if
 - The value for its first argument, `key`, is the symbolic constant `IPC_PRIVATE`, or
 - the value `key` is not associated with an existing shared memory identifier and the `IPC_CREAT` flag is set as part of the `shmflg` argument or
 - the value `key` is not associated with an existing shared memory identifier and the `IPC_CREAT` along with the `IPC_EXCL` flag have been set as part of the `shmflg` argument.

Creating a Shared Memory Segment

- As with previous IPC system calls for message queues and semaphores, the `ftok` library function can be used to generate a key value.
- The argument `size` determines the size in bytes of the shared memory segment.
- If we are using `shmget` to access an existing shared memory segment, `size` can be set to 0, as the segment size is set by the creating process.

Creating a Shared Memory Segment

- The last argument for `shmget`, `shmflg`, is used to indicate segment creation conditions (e.g., `IPC_CREAT`, `IPC_EXCL`) and access permissions (stored in the low order 9 bits of `shmflg`).
- At this time the system does not use the execute permission settings.
- To specify creation conditions along with access permissions, the individual items are bitwise ORed.

Creating a Shared Memory Segment

- The `shmget` system call does not entitle the creating process to actually use the allocated memory.
- It merely reserves the requested memory.
- To be used by the process, the allocated memory must be attached to the process using a separate system call.

Creating a Shared Memory Segment

- If `shmget` is successful in allocating a shared memory segment, it returns an integer shared memory identifier.
- If `shmget` fails, it returns a value of -1 and sets the value in `errno` to indicate the specific error condition.

Shared Memory Control

- The `shmctl` system call permits the user to perform a number of generalized control operations on an existing shared memory segment and on the system shared memory data structure.

Shared Memory Control

Include File(s)	<code><sys/ipc.h></code> <code><sys/shm.h></code>	Manual Section	2
Summary	<code>int shmctl(int shmid, int cmd, struct shmid_ds *buf);</code>		
Return	Success	Failure	Sets errno
	0	-1	Yes

Table 8.4. Summary of the shmctl System Call.

Shared Memory Control

- There are three arguments for the `shmctl` system call:
 - The first, `shmid`, is a valid shared memory segment identifier generated by a prior `shmget` system call.
 - The second argument, `cmd`, specifies the operation `shmctl` is to perform.
 - The third argument, `buf`, is a reference to a structure of the type `shmid_ds`.

Shared Memory Control

- If `shmctl` is successful, it returns a value of 0; otherwise, it returns a value of -1 and sets the value in `errno` to indicate the specific error condition.

Shared Memory Operations

- There are two shared memory operation system calls.
- The first, `shmat`, is used to attach (map) the referenced shared memory segment into the calling process's data segment.

(Table 8.6.)

Shared Memory Operations

Include File(s)	<code><sys/ipc.h></code> <code><sys/shm.h></code>	Manual Section	2
Summary	<code>void *shmat(int shmid, const void *shmaddr, int shmflg);</code>		
Return	Success	Failure	Sets errno
	Reference to the data segment	-1	Yes

Table 8.6. Summary of the `shmat` System Call.

Shared Memory Operations

- The first argument to `shmat`, `shmid`, is a valid shared memory identifier.
- The second argument, `shmaddr`, allows the calling process some flexibility in assigning the location of the shared memory segment.
 - If a nonzero value is given, `shmat` uses this as the attachment address for the shared memory segment.
 - If `shmaddr` is 0, the system picks the attachment address.
 - In most situations, it is advisable to use a value of 0 and have the system pick the address.

Shared Memory Operations

- The third argument, `shmflg`, is used to specify the access permissions for the shared memory segment and to request special attachment conditions, such as an aligned address or a read-only segment.
- The values of `shmaddr` and `shmflg` are used by the system to determine the attachment address.

Shared Memory Operations

- When `shmat` is successful, it returns the address of the actual attachment.
- If `shmat` fails, it returns a value of -1 and sets `errno` to indicate the source of the error.
- Remember that after a `fork`, the child inherits the attached shared memory segment(s).
- After an `exec` or an `exit` attached, shared memory segment(s) are detached but are not destroyed.

Shared Memory Operations

- The second shared memory operation, `shmdt`, is used to detach the calling process's data segment from the shared memory segment.

(Table 8.8.)

Shared Memory Operations

Include File(s)	<code><sys/types.h></code> <code><sys/shm.h></code>	Manual Section	2
Summary	<code>int shmdt (const void *shmaddr);</code>		
Return	Success	Failure	Sets errno
	0	-1	Yes

Table 8.8. Summary of the shmdt System Call

Shared Memory Operations

- The `shmdt` system call has one argument, `shmaddr`, which is a reference to an attached memory segment.
- If `shmdt` is successful in detaching the memory segment, it returns a value of 0.
- If the `shmdt` call fails, it returns a value of -1 and sets `errno`.

Example: Using semaphores and shared memory

The following two examples illustrate programs that support the client/server model.

Server program

This program acts as a server to the client program (see Client program). The buffer is a shared memory segment. The process synchronization is done using semaphores.

Client program

This program acts as a client to the server program (see Server program). The program is run after the message Ready for client jobs appears from the server program.